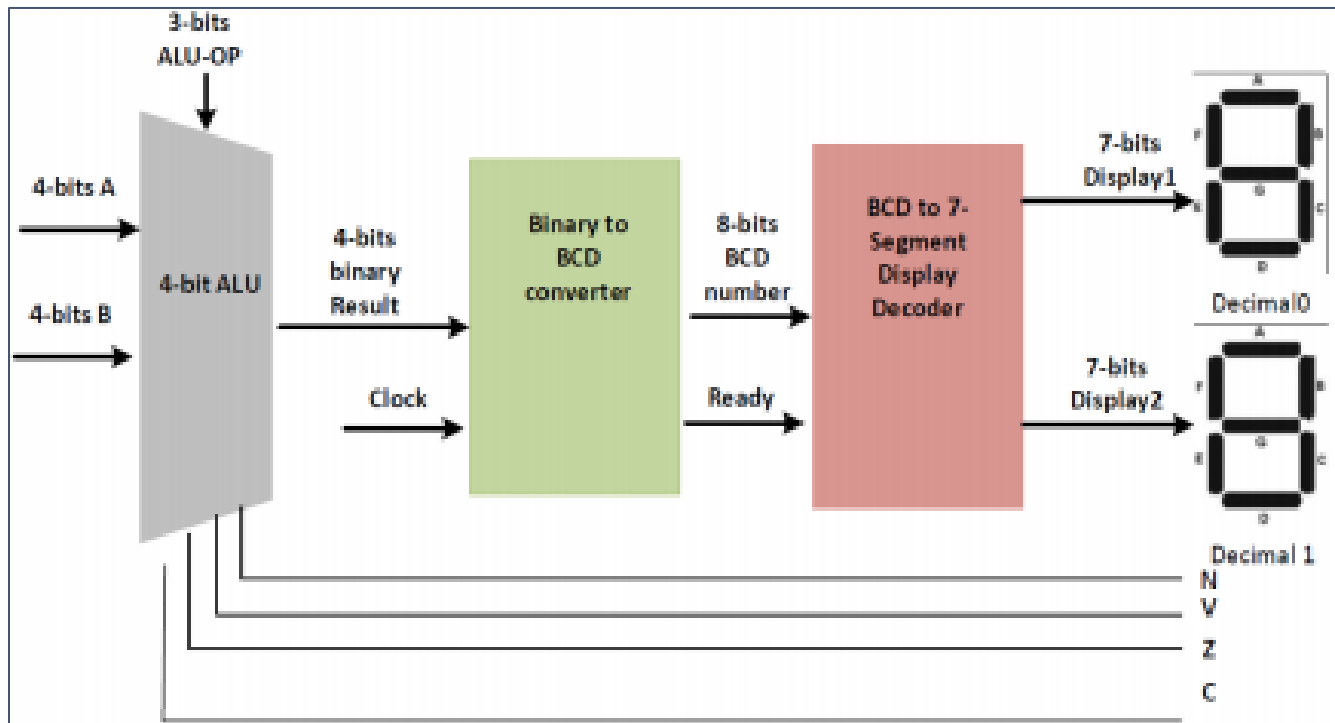


LAB 3: Vending Machine

Date: 29 May of 2025



Proffesor: Dr. Francisco Javier Rodríguez Navarrete

Students: Cesar Eduardo Inda Cenicerros

Alonso Emmanuel López Macías

Team: 14

Introduction

Finite State Machines (FSMs) are a fundamental tool in digital system design, allowing developers to model systems whose outputs depend not only on current inputs but also on previous states. In this laboratory practice, a vending machine was implemented using an FSM-based design with Verilog HDL. The system simulates a real-world embedded control scenario, where the vending machine accepts coins, manages product selection, and performs operations such as dispensing products, returning change, or canceling transactions.

The vending machine supports four different products, each with a unique cost, and is controlled through a set of inputs including buttons and switches. The system was implemented on an FPGA platform, where LEDs simulate motors, and 7-segment displays show user input, total amount inserted, and change returned. This project illustrates how sequential logic and FSMs can be used to coordinate a user-interactive digital system in real time, forming a strong foundation for embedded system development. This project highlights the importance of sequential logic in embedded system applications. By integrating counters, display drivers, and control logic, the vending machine responds in real time to user actions, ensuring accurate and safe operation. It also emphasizes the relevance of modular design and state-based thinking when implementing practical digital systems on FPGAs.

Through this lab, students gain hands-on experience with FSM modeling, resource mapping in FPGA hardware, and Verilog coding practices, all of which are critical skills in modern embedded and digital systems engineering

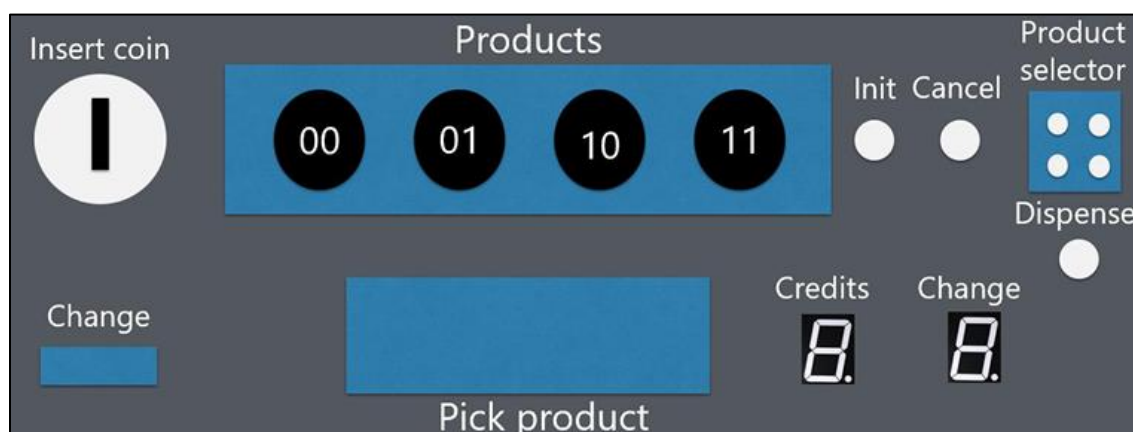


Figure: 1 Proposed For Vending Machine

Requirements

Design a state machine that models the behavior of a vending machine. The vending machine will have the following characteristics:

The vending machine offers four products with different prices. To begin a purchase, the user must press the **init button**, after which the machine will accept only **\$1 coins**, up to a maximum of **\$9**. The inserted amount is shown on a **7-segment display**. Once enough money is inserted, the user selects a product and presses the **dispense button** to receive it. If the inserted amount is **insufficient**, the machine will wait for either more coins or a cheaper product selection. If the amount is **greater**, the machine dispenses the product and shows the **change**. The **cancel button** allows the user to abort the purchase and retrieve the money. After completing or canceling the purchase, the machine waits for the user to **collect the product and change**, then resets to the initial state.

Producto	Costo
00	4
01	9
10	2
11	7

Figure: 2 Table Requeriments

The vending machine operates using a finite state machine (FSM) with four states:

- **IDLE**: The system is inactive.
- **INSERT_COIN**: Accepts coins.
- **DISPENSE**: Dispenses the selected product.
- **PICK_PRODUCT**: Product and change are available for collection.

The FSM is controlled through FPGA inputs such as buttons (to start, insert coins, dispense, or cancel), switches (to select products and simulate product/change collection), and LEDs (to simulate motors and indicate the current state). The state transitions and system behavior are illustrated in the FSM diagram.

The consideration for FSM is following:

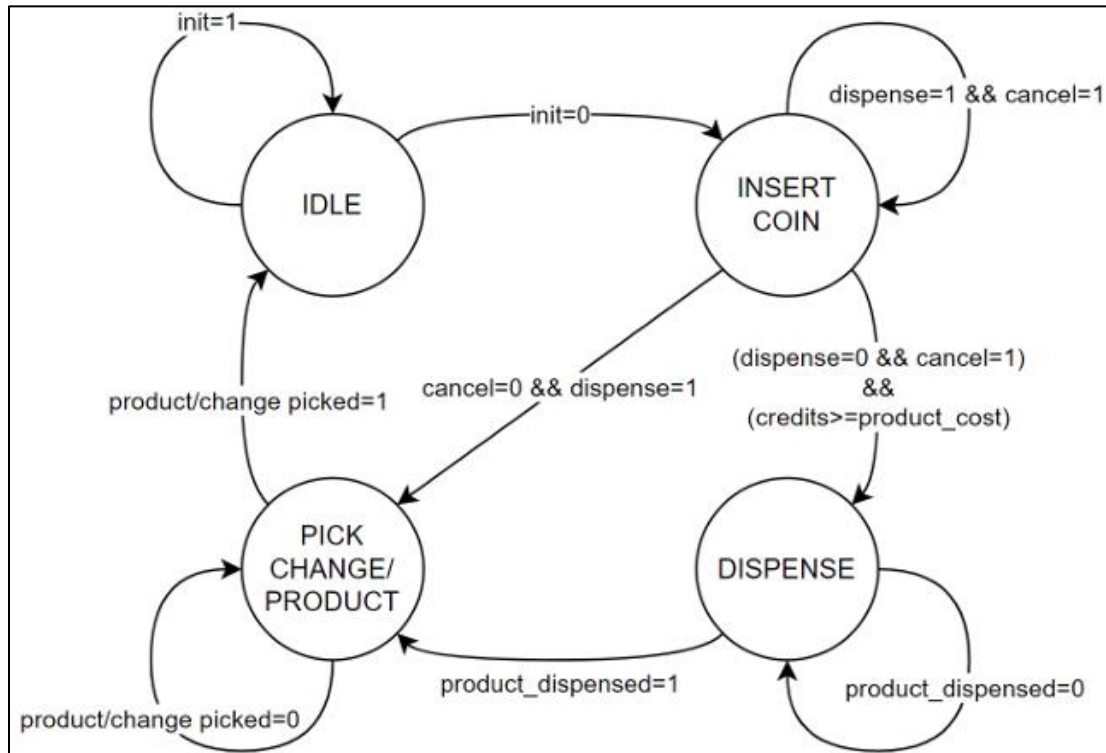


Figure: 3 FMS Vending Machine

EXTRA LOGIC

In addition to the main FSM, extra logic is required to handle specific actions in each state:

- **IDLE:** Reset all 7-segment displays to zero.
- **INSERT_COIN:** Use a counter to track inserted coins and display the total on a 7-segment display.
- **DISPENSE:** Activate the motor to release the product.
- **PICK_CHANGE/PRODUCT:** Calculate and display the change on a 7-segment display based on the previous transaction.

Development

As a first consideration, we established all the program logic in a single section due to the extensive nature of the functions. This practice was more challenging for us, which is why we decided to define it this way. In this first part, we defined the module in which we were going to create the multi-product machine with the assignments for each of the input and output elements.

```
module vending_multi_purchase (
    input wire clk,
    input wire rst,                // SW0

    input wire coin_btn,          // KEY1
    input wire cancel_btn,       // KEY2
    input wire dispense_btn,      // KEY3

    input wire [1:0] sel_product, // SW17 y SW16
    input wire product_dispensed_sw, // SW1

    output reg [6:0] display_credit, // HEX0
    output reg [6:0] display_change, // HEX1
    output reg [6:0] display_price,  // HEX2
    output reg motor_led,            // LEDG8
    output reg [1:0] state_leds      // LEDG0, LEDG1
);
```

In the other section we define a function for case converter to seven segments:

```
function [6:0] to7seg;
    input [3:0] value;
    begin
        case (value)
            4'd0: to7seg = 7'b1000000;
            4'd1: to7seg = 7'b1111001;
            4'd2: to7seg = 7'b0100100;
            4'd3: to7seg = 7'b0110000;
            4'd4: to7seg = 7'b0011001;
            4'd5: to7seg = 7'b0010010;
            4'd6: to7seg = 7'b0000010;
            4'd7: to7seg = 7'b1111000;
            4'd8: to7seg = 7'b0000000;
            4'd9: to7seg = 7'b0010000;
            default: to7seg = 7'b1111111;
        endcase
    end
endfunction

function [3:0] get_price;
    input [1:0] sel;
    begin
        case (sel)
            2'b00: get_price = 4;
            2'b01: get_price = 9;
            2'b10: get_price = 2;
            2'b11: get_price = 7;
            default: get_price = 0;
        endcase
    end
endfunction
```

This part defines all the logic for the functions in the converter to bcd's and we can observe the decimal values.

We create a “main logic principle” in this part we can choose a always positive sedge state for counter in the clock’s times. And each one condition represents a different state as an FSM, this section is critical in the functionality in the other key buttons and the final result in the bcd conditions. In the final we considerate change HEX2 → HEX6, because in visual representation is near from the selector option SW16 & SW17.

```

always @(posedge clk or posedge rst) begin
  if (rst) begin
    coin_count <= 0;
    change <= 0;
    motor_active <= 0;
  end else begin
    // Detectar flancos de bajada
    coin_btn_prev <= coin_btn;
    cancel_btn_prev <= cancel_btn;
    dispense_btn_prev <= dispense_btn;

    // Insertar moneda
    if (coin_btn_prev && ~coin_btn) begin
      if (coin_count < 9)
        coin_count <= coin_count + 1;
    end

    // Obtener precio del producto actual
    product_price <= get_price(sel_product);

    // Comprar producto si alcanza
    if (dispense_btn_prev && ~dispense_btn) begin
      if (coin_count >= product_price) begin
        motor_active <= 1;
        coin_count <= coin_count - product_price;
        change <= 0;
      end
    end

    // Producto entregado - apagar motor
    if (motor_active && product_dispensed_sw)
      motor_active <= 0;

    // Cancelar - devolver todo el crédito como cambio
    if (cancel_btn_prev && ~cancel_btn) begin
      change <= coin_count;
      coin_count <= 0;
      motor_active <= 0;
    end
  end
end

// ==== Salidas ====

always @(*) begin
  display_credit = to7seg(coin_count);
  display_change = to7seg(change);
  display_price = to7seg(product_price); // mostrar precio actual en HEX2
  motor_led = motor_active;
  state_leds = sel_product;
end

endmodule

```

Figure: 4 Fsm Conditions

Testbench

In this case, the **"tb_vending_machine"** is added in the "assignments" option. We define a customization for the products and the name company. We instantiated the DUT here:

```
timescale 1ns / 1ps

module tb_vending_machine;

    reg clk = 0;
    reg rst = 1;

    reg coin_btn = 0;
    reg cancel_btn = 0;
    reg dispense_btn = 0;
    reg product_dispensed_sw = 0;
    reg [1:0] sel_product = 2'b00;

    wire [6:0] display_credit;
    wire [6:0] display_change;
    wire [6:0] display_price;
    wire motor_led;
    wire [1:0] state_leds;

    // Instanciar el DUT
    vending_multi_purchase uut (
        .clk(clk),
        .rst(rst),
        .coin_btn(coin_btn),
        .cancel_btn(cancel_btn),
        .dispense_btn(dispense_btn),
        .sel_product(sel_product),
        .product_dispensed_sw(product_dispensed_sw),
        .display_credit(display_credit),
        .display_change(display_change),
        .display_price(display_price),
        .motor_led(motor_led),
        .state_leds(state_leds)
    );

endmodule
```

Figure: 5 Testbench_DUT

Then add the initialization for conditions and display monitor messages:

```
initial begin
    // Mensaje de bienvenida
    $display("\nHola, Bienvenido a Machine'KFC'");
    $display("El precio de los productos es:");
    $display("00: Papas ..... $2");
    $display("01: Combo Mcchiken ..... $9");
    $display("10: Nieve ..... $4");
    $display("11: Platicillo individual .. $7\n");

    // Encabezado de monitoreo
    $display("Tiempo | Credito | Cambio | Motor | Producto");
    $monitor("%5dns | %b | %b | %b | %b",
        $time, display_credit, display_change, motor_led, sel_product);

    // Reset
    #20 rst = 0;

    // Insertar $3
    repeat (3) begin
        #10 coin_btn = 1; #10 coin_btn = 0;
    end

    // Seleccionar papas (producto 10, $2)
    #10 sel_product = 2'b10;
    #10 dispense_btn = 1; #10 dispense_btn = 0;
    #10 product_dispensed_sw = 1; #10 product_dispensed_sw = 0;

    // Seleccionar nieve (producto 00, $4), saldo restante: $1
    #10 sel_product = 2'b00;
    #10 dispense_btn = 1; #10 dispense_btn = 0;

    // Insertar $1 más
    #10 coin_btn = 1; #10 coin_btn = 0;

    // Intentar nuevamente
    #10 dispense_btn = 1; #10 dispense_btn = 0;
    #10 product_dispensed_sw = 1; #10 product_dispensed_sw = 0;

    // Cancelar para recibir cambio
    #10 cancel_btn = 1; #10 cancel_btn = 0;

    #50 $finish;
end

endmodule
```

Figure: 6 Testbench Conditions

We can observe in **ModelSim simulation** the following:

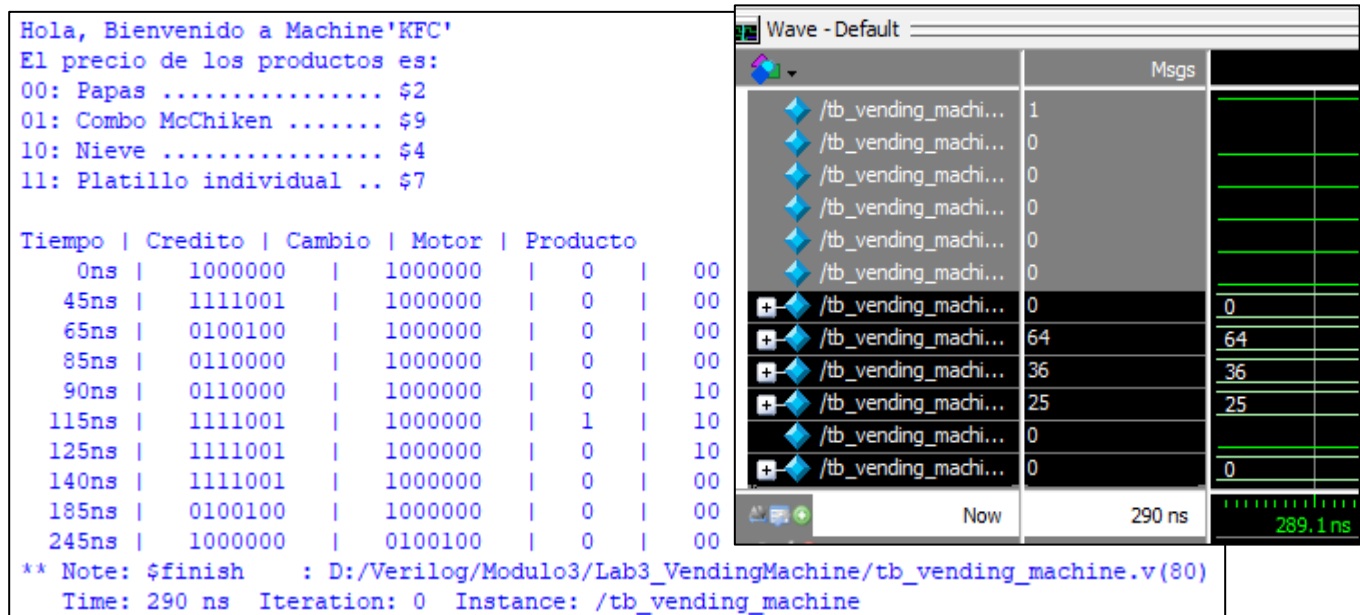


Figure: 7 Testbench_Display

FPGA Implementation

Once the program was completed, the compilation process was carried out in order to generate the pin assignment plan and correctly map the pins according to the program's requirements.

Flow Status	Successful - Sun Jun 01 13:38:29 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	lab3_vending_m
Top-level Entity Name	vending_multi_purchase
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	48 / 114,480 (< 1 %)
Total registers	16
Total pins	32 / 529 (6 %)
Total virtual pins	0
Total memory bits	0 / 3,981,312 (0 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)

Figure: 8 Compilation Report

The pins consideration was assigned as following, in this case we add different elements for complete the functionality in the FPGA.

in	cancel_btn	Input	PIN_N21
in	clk	Input	PIN_Y2
in	coin_btn	Input	PIN_M21
in	dispense_btn	Input	PIN_R24
out	display_change[6]	Output	PIN_U24
out	display_change[5]	Output	PIN_U23
out	display_change[4]	Output	PIN_W25
out	display_change[3]	Output	PIN_W22
out	display_change[2]	Output	PIN_W21
out	display_change[1]	Output	PIN_Y22
out	display_change[0]	Output	PIN_M24
out	display_credit[6]	Output	PIN_H22
out	display_credit[5]	Output	PIN_J22
out	display_credit[4]	Output	PIN_L25
out	display_credit[3]	Output	PIN_L26
out	display_credit[2]	Output	PIN_E17
out	display_credit[1]	Output	PIN_F22
out	display_credit[0]	Output	PIN_G18

out	display_price[6]	Output	PIN_AC17
out	display_price[5]	Output	PIN_AA15
out	display_price[4]	Output	PIN_AB15
out	display_price[3]	Output	PIN_AB17
out	display_price[2]	Output	PIN_AA16
out	display_price[1]	Output	PIN_AB16
out	display_price[0]	Output	PIN_AA17
out	motor_led	Output	PIN_F17
in	product...nsed_sw	Input	PIN_AC28
in	rst	Input	PIN_AB28
in	sel_product[1]	Input	PIN_Y23
in	sel_product[0]	Input	PIN_Y24
out	state_leds[1]	Output	PIN_E22
out	state_leds[0]	Output	PIN_E21

Figure: 9 PinPlanner

The RTL view can be observed as follows.

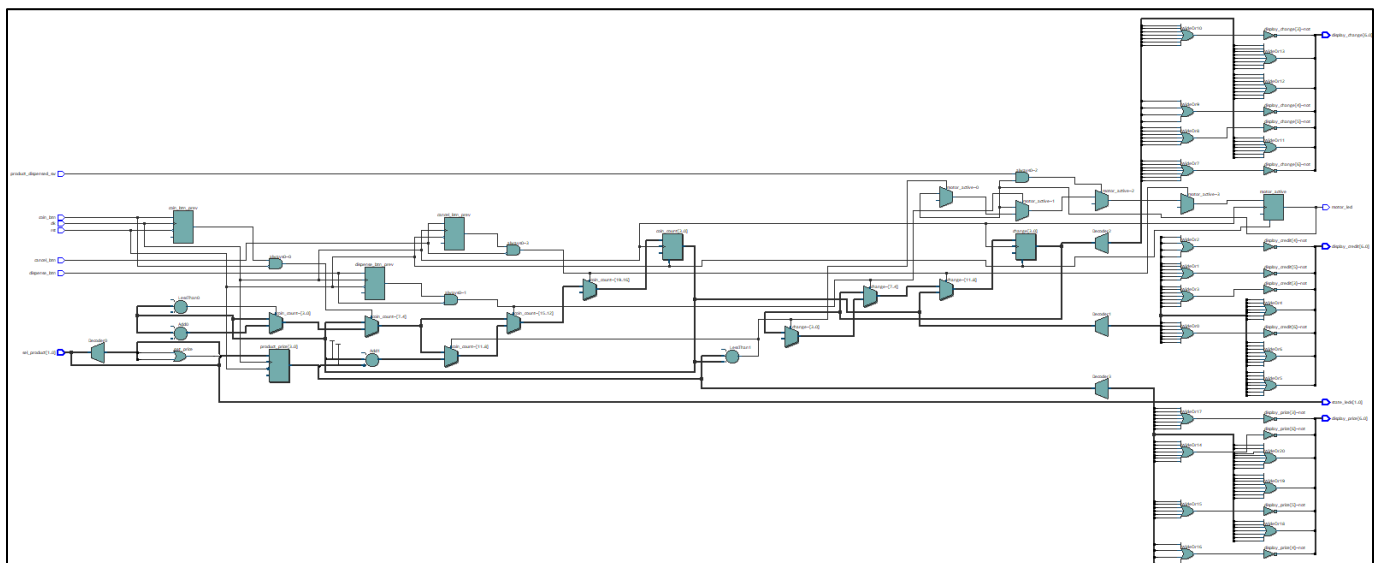


Figure: 10 RTL Viewer

FPGA Results

To perform this FPGA we can simulate a 3 buys in each one we set a \$9 credit value and continue shopping the products until we have a credit that is not enough for buy other product, with this we can observe that the functionality is correct the LEDG8 represent the motor in state on because the machine continues dispenser the products and continues with a enough value for other product.

1.- Product = \$4 , Credit \$9 , Change = \$5 → Continuation we can continue buy other product

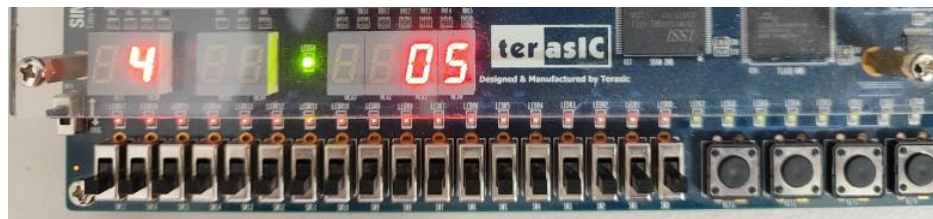


Figure: 11 Test1

2.- Product = \$2 , Credit \$5 , Change = \$3 → Continuation we can continue buy other product



Figure: 12 Test2

3.- Product = \$2 , Credit \$3 , Change = \$1 → Continuation we can continue buy other product



Figure: 13 Test3

GitHub Repository

https://github.com/CeicGitHub/Fundamentals Of Digital Design FPGA 3/tree/Lab3_VendingMachine

Conclusion / Issues

Cesar Eduardo Inda Cenicerros

I believe that in this lab we faced quite a few issues, since in order to implement multiple validations, we had to start separating the logic. First, we made sure that the credit amount worked and that the purchase could be canceled. Then, we added the selector part with the products and made sure they were displayed correctly. Once each functionality was working, we combined the logic to ensure that, when pressing the buttons, the appropriate calculations would be performed by the program. It was challenging to consider certain elements, as the FSM represented a more complex challenge compared to other practices.

Alonso Emmanuel Lopez Macias

We had to segment much of the implementation to ensure it was working correctly. On many occasions, the product purchase would go through, but when trying to buy another product, the value stored in the credit variable would reset to 0. So, we had to start adding many conditions that would allow us to retrieve the remaining credit to continue purchasing. Part of the lab, especially the testbench, also posed difficulties when dealing with the conditions we had created, since some initial considerations triggered errors due to unsupported elements, we had previously been working with in System Verilog. In this case, using Verilog made it significantly more complex. I learned a lot from this lab, but I believe the difficulty level was quite high.